

5팀 최종발표

입찰메이트 - RFP 문서 RAG 시스템 개발

팀장 이승철

팀원 김동현

이경식

최경운

01	프로젝트 목표 확인
02	데이터 확인 및 분석
03	프로젝트 환경 구성
04	프로젝트 목표에 맞는 기초 RAG 시스템 개발과정
05	멀티모달 로더를 활용한 RAG 시스템 개발
06	RAG 시스템과 FRONT END 연결
07	CRAG를 적용한 아키텍처 소개
08	시스템 성능 평가

1. 프로젝트 목표 확인

본 프로젝트는 자연어 처리 및 LLM 기반 RAG 시스템을 구축하여, 복잡하고 분량이 많은 기업·정부 제안요청서(RFP)에서 핵심 정보를 빠르게 추출·요약하고 질의응답을 제공하는 서비스를 만드는 것을 목표로 합니다.

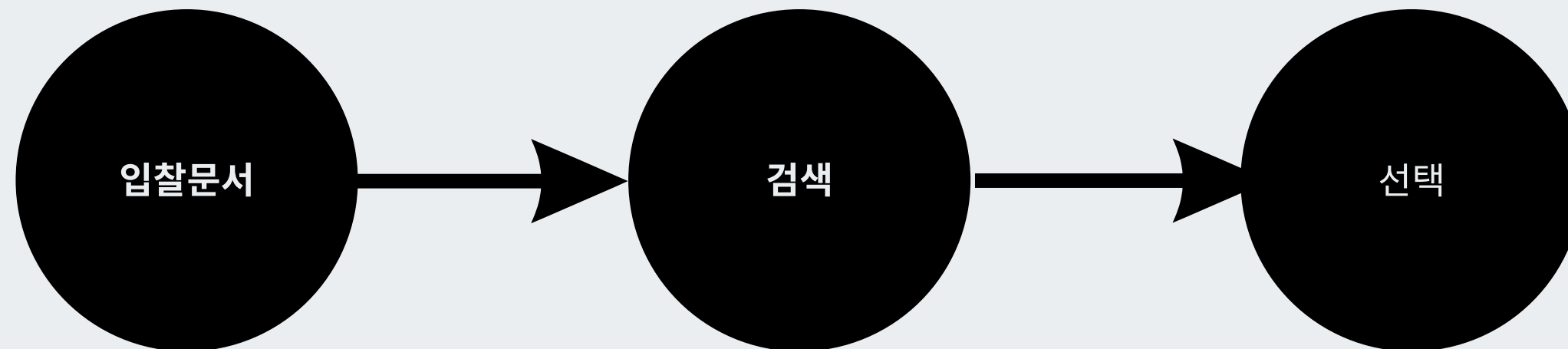
B2G 입찰 컨설팅 스타트업 '입찰메이트'의 엔지니어링 팀 역할을 맡아, 하루 수백 건씩 쏟아지는 RFP 문서 속에서 사업 요구 사항, 발주기관, 예산, 제출 방식 등 주요 정보를 자동으로 파악할 수 있는 사내 RAG 시스템을 구현합니다.

이를 통해 컨설턴트들은 수십 페이지의 문서를 직접 읽지 않고도, 고객사에 적합한 입찰 기회를 빠르게 선별하고 컨설팅 업무에 집중할 수 있게 됩니다.

기존 방식



신규 방식



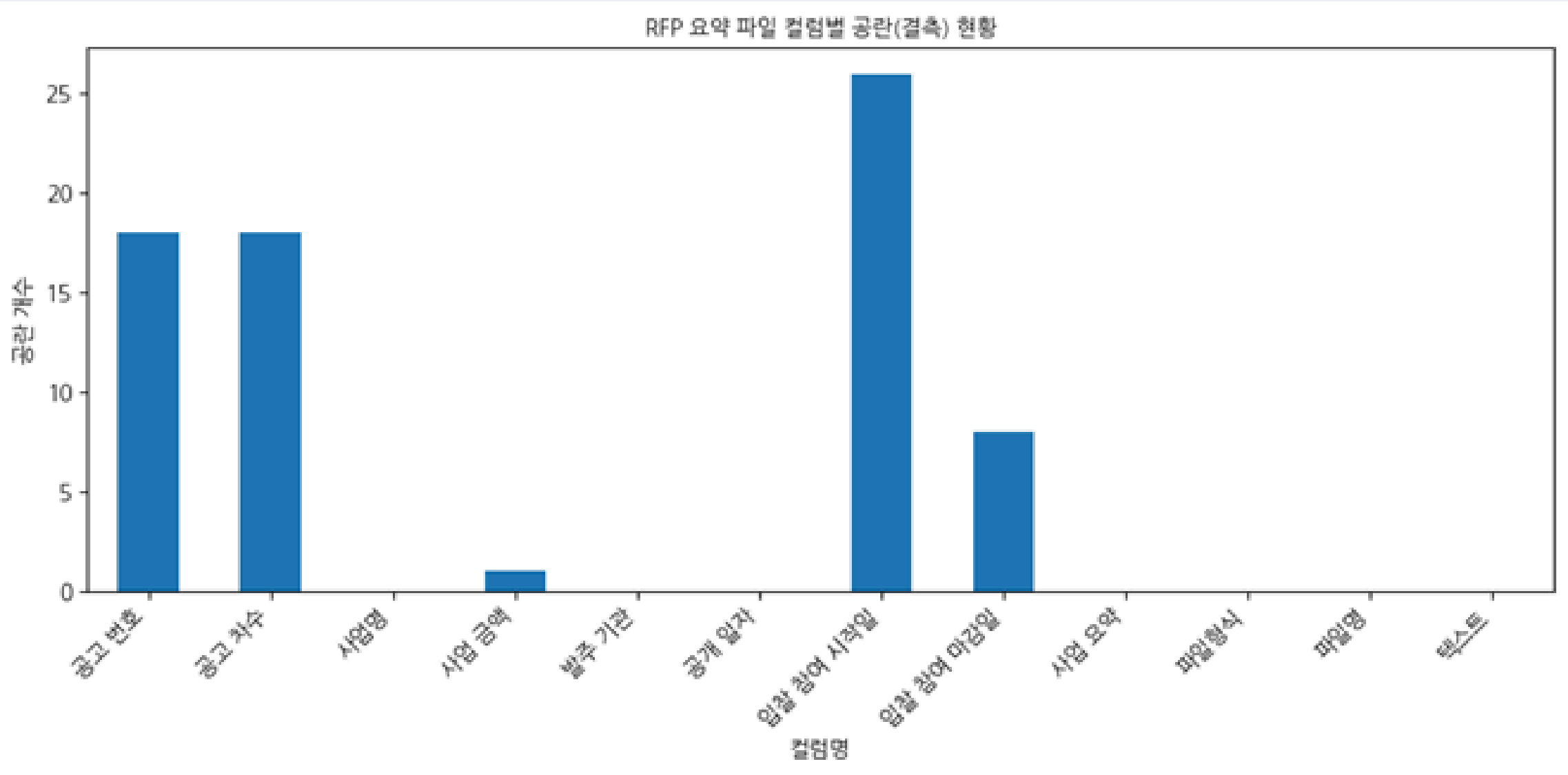
2. 데이터 확인 및 분석

원본 데이터

- 총 100개의 제안 요청서
 - PDF 파일 4개, HWP 파일 96개
- 제안 요청서 내 주요 정보가 요약된 csv 파일 1개

CSV 파일

- CSV 파일 내 항목 별 누락된 값의 개수 측정
- 텍스트 항목은 누락된 값은 없지만, 저장되어 있는 텍스트 내용이 파일의 일부분에 불과함
- 텍스트 외 데이터들을 메타데이터로서 활용할 수는 있지만 RAG에 사용할 수 있는 텍스트 데이터의 품질은 좋지 못함



CSV 파일 결측치 확인

3. 프로젝트 환경 구성

GCP 서버 구성

- 디스크 용량 50GB 설정
- 팀원 별 아이디 및 비밀번호 할당

VSCode 연동

- VSCode에서 SSH 원격 접속을 통해 서버 상에서 작업할 수 있도록 SSH 키 연동
- 서버 외부 IP와 암호 키 등록을 통해 VSCode에서 GCP 서버에 원격 접속
- 이를 통해 개인 IDE에서 서버에 데이터, 코드를 추가하거나 수정할 수 있는 환경 구성

Github 원격 저장소 연동

- 서버에서 생성한 SSH 키를 Github에 등록
- 서버 상에서 Github 저장소를 자유롭게 복제하거나, 저장소를 업데이트할 수 있는 환경 구성

3. 프로젝트 환경 구성시 발생 이슈

1. 디스크 용량 부족 이슈

- RFP 원본 문서(PDF), 전처리 텍스트, 벡터 DB, 팀원 별 시스템 구성 등으로 인해 초기 할당 디스크 용량이 빠르게 소진됨
- GCP 디스크 용량 증설, 통합 시스템 활용 및 불필요한 중간 산출물 정리로 해결(50GB->100GB)

2. 원격 SSH 연결 설정 이슈

- 초기 GCP 인스턴스 생성 후 VS Code Remote SSH 연결이 원활하지 않음
- 방화벽 확인 및 SSH 키 설정, 포트 확인을 통해 안정적인 원격 접속 환경 구축

3. 간헐적인 원격 연결 끊김 현상

- 장시간 작업 시 SSH 세션이 끊기는 문제가 발생
- GCP 서버의 문제로 상업적인 활용 시 유료 서비스 상태 확인 및 AWS 같은 타 서비스 활용 필요

4. 기초 RAG 시스템 개발

데이터 확인

Loader(로더):

PDF HWP 포맷의 RFP 문서를
텍스트로 읽어들이м

벡터화

Text Splitter(스플리터):

긴 문서를 의미 있는 단위
(Chunk)로 쪼개어 검색 효율성 증
대.

Embedding Model(임베딩):

텍스트를 컴퓨터가 이해할 수 있는
수치(벡터)로 변환.

Vector Database(Chroma)

변환된 벡터 데이터를 저장하고 관
리

검색

Retriever(검색기)

질문과 가장 유사한 문서 조각을
DB에서 찾아냄

답변생성

Prompt(프롬프트)

LLM에게 "RFP 전문가로서 답변
하라"는 페르소나와 검색된 지문을
전달.

LLM

최종적으로 맥락을 파악하여 자연
스러운 답변 생성.

```
from pathlib import Path
from typing import List

from langchain_core.documents import Document
from .base import BaseRFPDocumentLoader

from langchain_community.document_loaders import PyMuPDFLoader
from helper_hwp import hwp_to_txt


class PyMuPDFHwpRFPDocumentLoader(BaseRFPDocumentLoader):
    def load(self, path: str | Path) -> List[Document]:
        path = Path(path)
        if path.suffix.lower() == ".pdf":
            loader = PyMuPDFLoader(str(path))
            docs = loader.load()
        elif path.suffix.lower() == ".hwp":
            txts = hwp_to_txt(str(path))
            docs = [Document(page_content=txts)]
        else:
            raise ValueError(f"지원하지 않는 확장자: {path.suffix}")

        # 공통 메타데이터 추가 (RFP 프로젝트용)
        for d in docs:
            d.metadata.setdefault("source", str(path))
            d.metadata.setdefault("doc_type", "rfp")
        return docs

    def load_directory(self, dir_path: str | Path) -> List[Document]:
        dir_path = Path(dir_path)
        all_docs: List[Document] = []
        for fp in dir_path.glob("**/*"):
            if fp.suffix.lower() in [".pdf", ".hwp"]:
                all_docs.extend(self.load(fp))
        return all_docs
```

- load 메소드에서는 확장자(.hwp, .pdf)별 다른 패키지를 사용해 파일 로드
- load_directory 메소드에서 원본 데이터가 저장되어 있는 폴더를 순회하며 폴더 내에 있는 각 파일을 하나씩 로드함(load 메소드 호출)

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_core.documents import Document
from typing import List
from ..config import get_app_config

def get_text_splitter(
    chunk_size: int = 1000,
    chunk_overlap: int = 150,
) -> RecursiveCharacterTextSplitter:
    return RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
        separators=["\n\n", "\n", " ", ""],
    )

def split_documents(docs: List[Document]) -> List[Document]:
    cfg = get_app_config()
    splitter = get_text_splitter(cfg.chunking.chunk_size, cfg.chunking.chunk_overlap)
    return splitter.split_documents(docs)
```

- langchain에서 제공하는 기본 텍스트 스플리터인 RecursiveCharacterTextSplitter 사용

OpenAI

```

from langchain_openai import OpenAIEmbeddings
from ..config import get_app_config

def get_openai_embeddings():
    """
    OpenAI 임베딩 모델의 인스턴스를 가져옵니다.
    Returns:
    | OpenAIEmbeddings: OpenAI 임베딩 모델 객체
    """
    cfg = get_app_config()
    return OpenAIEmbeddings(
        model=cfg.embeddings.model_name,
        api_key=cfg.model_api_key,
    )

```

```

p_config
import get_local
import get_openai

```

```

al_hf":
    _embeddings()
penai_api":
    mbeddings()

```

지원하지 않는 F

HuggingFaceEmbeddings

```

from langchain_huggingface import HuggingFaceEmbeddings
from ..config import get_app_config

def get_local_hf_embeddings():
    """
    HuggingFace 임베딩 모델을 로드하여 반환합니다.
    Returns:
    | HuggingFaceEmbeddings: 로컬 HuggingFace 임베딩 모델 객체
    """
    cfg = get_app_config()
    return HuggingFaceEmbeddings(
        model_name=cfg.embeddings.model_name,
        model_kwargs={
            "device": cfg.device,
            # jinaai
            "trust_remote_code": True,
            "token": cfg.model_api_key,
        },
        encode_kwargs={"normalize_embeddings": True},
    )

```

- 설정에 따라 OpenAI의 임베딩 모델을 사용할지 HuggingFace의 임베딩 모델을 사용할 지 결정하는 로직 구현

```
from langchain_core.embeddings import Embeddings
from ..config import get_app_config
from pathlib import Path
from typing import List
from langchain_core.documents import Document
```

```
def create_chroma_from_documents(
    docs: List[Document],
    embeddings: Embeddings,
    collection_name: str = "rfp_rag",
) -> Chroma:
```

```
    """
```

```
    주어진 문서들로 Chroma 벡터저장소를 생성하고 저장합니다.
```

```
    Args:
```

```
        docs: Document 목록
        embeddings: 임베딩 모델
        collection_name: 컬렉션 이름
```

```
    Returns:
```

```
        Chroma 벡터저장소
```

```
    """
```

```
    cfg = get_app_config()
```

```
    # 벡터저장소 디렉토리 없을시 생성
```

```
    persist_dir = Path(cfg.vectorstore.persist_dir)
```

```
    persist_dir.mkdir(parents=True, exist_ok=True)
```

```
    # 입력받은 문서로 벡터저장소 생성
```

```
    vectordb = Chroma.from_documents(
        documents=docs,
        embedding=embeddings,
        collection_name=collection_name,
        persist_directory=str(persist_dir),
    )
```

```
    return vectordb
```

```
from pathlib import Path
from ..loaders import get_document_loader
from ..chunking.splitter import split_documents
from ..embeddings import get_embeddings
from ..vectorstores.chroma_store import create_chroma_from_documents
```

```
def ingest_documents(source_dir: str | Path):
```

```
    loader = get_document_loader()
```

```
    print(f"[INGEST] Loading documents from {source_dir} ...")
```

```
    docs = loader.load_directory(source_dir)
```

```
    print(f"[INGEST] Loaded {len(docs)} documents. Splitting...")
```

```
    chunks = split_documents(docs)
```

```
    print(f"[INGEST] Created {len(chunks)} chunks.")
```

```
    embeddings = get_embeddings()
```

```
    print("[INGEST] Creating Chroma vectorstore...")
```

```
    vectordb = create_chroma_from_documents(chunks, embeddings)
```

```
    print("[INGEST] Done. Vectorstore persisted.")
```

```
    return vectordb
```

- ingest_documents는 정의한 로더, 텍스트 스플리터, 임베딩 모델을 불러와서 벡터 DB가 없는 경우 원본 데이터가 저장된 폴더를 읽어 벡터 DB를 새로 생성하는 메소드


```
def load_chroma(
    embeddings: Embeddings,
    collection_name: str = "rfp_rag",
) -> Chroma:
    """
    폴더에 저장된 Chroma 벡터저장소를 로드합니다.
    Args:
        embeddings: 임베딩 모델
        collection_name: 컬렉션 이름
    Returns:
        Chroma 벡터저장소
    """
    cfg = get_app_config()
    # 폴더에 저장된 벡터저장소 로드
    return Chroma(
        collection_name=collection_name,
        embedding_function=embeddings,
        persist_directory=cfg.vectorstore.persist_dir,
    )
```

```
from ..embeddings import get_embeddings
from ..vectorstores.chroma_store import load_chroma

def get_retriever(k: int = 5):
    embeddings = get_embeddings()
    vectordb = load_chroma(embeddings)
    retriever = vectordb.as_retriever(search_kwargs={"k": k})
    return retriever
```

- load_chroma는 벡터 DB가 이미 존재하는 경우 벡터 DB가 저장된 폴더를 읽어 벡터 DB를 불러오는 메소드
- get_retriever 메소드로 읽어온 벡터 DB에서 검색기를 정의

OpenAI

```
from langchain_openai import ChatOpenAI
from ..config import get_app_config

def get_openai_llm():
    """
    OpenAI LLM을 생성합니다.
    Returns:
        ChatOpenAI: OpenAI LLM 객체
    """
    cfg = get_app_config()
    return ChatOpenAI(
        model=cfg.llm.model_name,
        api_key=cfg.model_api_key,
        temperature=cfg.llm.temperature,
        max_tokens=cfg.llm.max_new_tokens,
    )
```

```
...config
...t_local_
...openai_1

...ll_hf":
...llm()
...enai_api
...m()
```

지원하지 않

HuggingFaceEmbeddings

```
from transformers import AutoModelForCausalLM, AutoTokenizer, pipeline
from langchain_huggingface import HuggingFacePipeline
from ..config import get_app_config
from dotenv import load_dotenv

def get_local_hf_llm():
    """
    HuggingFacePipeline을 사용하여 로컬 LLM을 가져옵니다.
    Returns:
        HuggingFacePipeline: 로컬 HuggingFace LLM 객체
    """
    load_dotenv() # 루트 .env 로드
    cfg = get_app_config()
    model_name = cfg.llm.model_name

    tokenizer = AutoTokenizer.from_pretrained(model_name, token=cfg.model_api_key)
    model = AutoModelForCausalLM.from_pretrained(
        model_name,
        device_map=cfg.device,
        dtype="auto",
        # LGAI-EXAONE
        token=cfg.model_api_key,
        trust_remote_code=True,
    )
    gen_pipeline = pipeline(
        "text-generation",
        model=model,
        tokenizer=tokenizer,
        max_new_tokens=cfg.llm.max_new_tokens,
        do_sample=True,
        temperature=cfg.llm.temperature,
        top_p=0.9,
    )
    llm = HuggingFacePipeline(pipeline=gen_pipeline)
    return llm
```

검색 및 답변생성

```
from langchain_core.runnables import RunnablePassthrough
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

from ..llms import get_llm
from .retrieval import get_retriever

def build_rag_chain(k: int = 5):
    llm = get_llm()
    retriever = get_retriever(k=k)

    prompt = ChatPromptTemplate.from_template(
        """
        당신은 기업 및 정부 제안요청서(RFP)를 분석하는 도우미입니다.
        아래 제공된 문서 컨텍스트를 바탕으로 사용자의 질문에 한국어로 답변하세요.
        필요하다면 목록/표 형태로 요약하되, 근거가 없으면 추측하지 말고 "문서에 정보가 없음"이라고 답하세요.

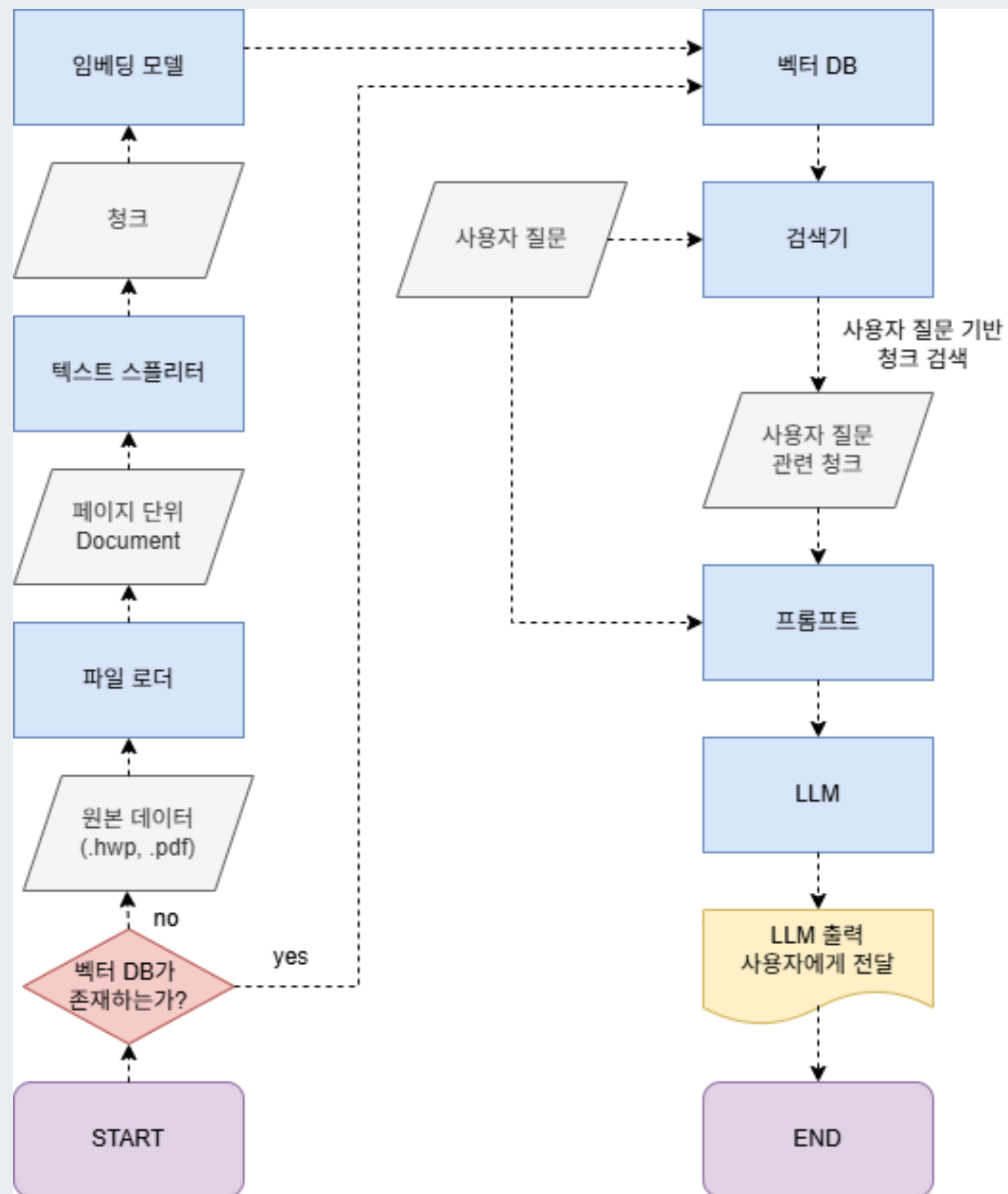
        # 질문:
        {question}

        # 참조 문서:
        {context}
        """
    )

    rag_chain = (
        {
            "context": retriever
            | (lambda docs: "\n\n".join(d.page_content for d in docs)),
            "question": RunnablePassthrough(),
        }
        | prompt
        | llm
        | StrOutputParser()
    )
    return rag_chain
```

- 벡터 DB에서 초기화한 검색기와 LLM 모델, 프롬프트를 사용해 RAG 체인을 구성
- RAG 체인의 invoke 메소드를 사용해 사용자의 질문 쿼리를 입력하면 자동으로 사용자의 질문이 검색기와 프롬프트에 입력됨
- 프롬프트에 검색기가 검색한 청크와 사용자의 질문이 입력되어 프롬프트가 LLM의 입력으로 전달됨
- LLM은 입력받은 프롬프트를 기반으로 출력을 생성

기초 RAG 시스템 아키텍처



env, yaml 파일

```
# LangSmith
LANGCHAIN_API_KEY=

# ChromaDB 저장 폴더
VECTORSTORE_PERSIST_DIR=

# 이미지 처리 결과 저장 폴더
IMAGE_OUTPUT_DIR=

# 원본 데이터 저장 폴더
RAW_DATA_DIR=

# 시나리오 1: 로컬 HF
HF_API_KEY=
HF_HOME=
HF_MODEL_NAME=LGAI-EXAONE/EXAONE-3.5-2.4B-Instruct
HF_EMBEDDING_MODEL=jinaai/jina-embeddings-v3
DEVICE=cuda # cpu 또는 cuda

# 시나리오 2: OpenAI API
OPENAI_API_KEY=
OPENAI_MODEL_NAME=gpt-5-mini
OPENAI_EMBEDDING_MODEL=text-embedding-3-small
```

```
configs > ! base.yaml
1 rag_mode: openai_api # local_hf 또는 openai_api
2
3 loader_config:
4   extract_tables: true
5   max_pages: 200
6   image_processing:
7     extract_images: true
8     caption:
9       enabled: true
10      model: "gpt-5-mini" # 비전 입력을 지원하는 모델로 설정
11      prompt_ko: |
12        당신은 RFP 문서 내에 포함된 이미지의 내용을 설명해주는 전문가입니다.
13        아래 지침에 따라 이미지를 분석하고 500자 이내로 요약해 주세요.
14        ---
15        - 도표/표/흐름도/아키텍처 그림이라면 핵심 구성요소(항목/축/범례/단계)와 의미를 요약해 주세요.
16        - 그래프/차트라면 어떤 데이터가 어떻게 변화하는지 요약 설명해 주세요.
17        - 기관 로고, 디자인 로고 등 그 외 이미지는 설명하지 말고 "RFP 문서와 관련이 없는 그림입니다."라고 답변해 주세요.
18        - 개인정보가 포함되어 있다면 설명에 포함하지 마세요.
19        ---
20
21 chunking:
22   chunk_size: 1000
23   chunk_overlap: 150
24
25 retrieval:
26   k_text: 3
27   k_table: 2
28   k_image: 2
29
30 vectorstore:|
31   collection_name: "rfp_rag"
32
33 llm:
34   temperature: 0.2
35   max_new_tokens: 2048
36
37 langsmith:
38   enabled: "true"
39   project: "rfp-rag-project"
```

설정 유지 클래스

```
class AppConfig(BaseModel):
    """
    애플리케이션의 설정을 담은 Pydantic 모델입니다.
    YAML 파일과 .env를 합쳐서 사용합니다.
    """

    rag_mode: str = "openai_api" # local_hf 또는 openai_api
    model_api_key: Optional[str] = None
    device: Optional[str] = None # "cuda" 또는 "cpu"
    raw_data_dir: str = None

    chunking: ChunkingConfig = Field(default_factory=ChunkingConfig)
    retrieval: RetrievalConfig = Field(default_factory=RetrievalConfig)
    vectorstore: VectorStoreConfig = Field(default_factory=VectorStoreConfig)
    loader_config: MultiModalLoaderConfig = Field(
        default_factory=MultiModalLoaderConfig
    )
    llm: LLMConfig = Field(default_factory=LLMConfig)
    embeddings: EmbeddingsConfig = Field(default_factory=EmbeddingsConfig)

    langsmith: LangSmithConfig = Field(default_factory=LangSmithConfig)
```

```
# ----- 외부에서 사용하는 진입점 -----

_config_cache: Optional[AppConfig] = None

def get_app_config() -> AppConfig:
    """
    YAML(base + profile) + .env를 합쳐 최종 AppConfig를 반환.
    (한 번 로드한 뒤 캐시)
    """
    global _config_cache
    if _config_cache is not None:
        return _config_cache
    yaml_config = load_yaml_if_exists(CONFIG_DIR / "base.yaml")

    # -----
    # yaml 설정 로드 및 반영
    # -----

    config = AppConfig(**yaml_config)

    # -----
    # .env / 환경변수 반영
    # -----

    config = set_config(config)
    _config_cache = config
    return config
```

- get_app_config 메소드를 실행시키면 설정이 저장된 yaml 파일과 .env 파일을 읽어 설정을 유지하는 클래스 객체에 저장 후 캐싱
- 캐싱된 설정 객체를 get_app_config 메소드 호출시마다 리턴하게 됨

5. 멀티모달 로더를 활용한 RAG 시스템 개발

멀티모달 로더의 개발 배경

기존에 사용하던 PDF 및 HWP 기반 텍스트 로더는 문서 내 텍스트 정보만 추출 가능하며, RFP 문서에 포함된 이미지와 표(Table) 영역을 처리하지 못하는 한계를 발견. 그러나 실제 입찰 제안 요청서 에서는 제출 서류 양식, 사업비 내역, 품질 및 기술 스펙과 같은 입찰 판단에 핵심적인 정보가 이미지나 표 형태로 제공되는 경우가 많아, 텍스트 중심의 로딩 방식으로는 정보 누락이 발생할 수밖에 없어, 멀티모달 로더 개발이 필요하다고 판단하여 개발을 진행

구분	사업금액의 하한
매출액 8천억원 이상인 대기업	80억원 이상
매출액 8천억원 미만인 대기업	40억원 이상
「중견기업 성장촉진 및 경쟁력 강화에 관한 특별법」 제2조의 '중견기업'이 된지 5년 이내 기업	20억원 이상

보안 위약금 부과 기준				
1. 위규 수준별로 A~D 등급으로 차등 부과				
구분	위규 수준			
	A급	B급	C급	D급
위규	심각 1건	중대 1건	보통 2건 이상	경미 3건 이상
위약금 비중	부정당업자 등록	총 사업비의 2%	총 사업비의 1%	총 사업비의 0.5%

해결 방안

본 프로젝트에서는 이런 문제를 해결하기 위해 텍스트·이미지·표 정보를 함께 처리할 수 있는 멀티모달 로더를 개발하고, 이를 RAG 시스템에 적용. 이를 통해 현재 제공된 100개의 RFP 문서뿐만 아니라, 향후 추가될 다양한 형식의 문서에 대해서도 일관되고 정확한 정보 추출이 가능하도록 설계.

입찰 참가 신청서				처리기간
※아래사항 중 해당되는 경우에만 기재하시기 바랍니다.				즉 시
신청인	상호/법인명칭		법인등록번호	
	주 소		전 화 번 호	
	대 표 자		주민등록번호	
입찰개요	입찰공고번호	제 2021 - 호	사업설명회 일 자	
	입찰건명			
입찰보증금	납 부	-보증금율: % -보증액: 금 정(W)		
	납부연제 및 지급확약	-사유 : -본인은 낙찰 후 계약 미체결 시 귀 대학에 낙찰금액에 해당하는 소정의 입찰보증금을 현금으로 납부할 것을 약속합니다. (인)		
대리인 · 사용인 · 감	본 입찰에 관한 일체의 권한을 다음의 자에게 위임합니다.		본 입찰에 사용할 인감을 다음과 같이 신고합니다.	
	성 명 : 주민등록번호 :		사용인감 : (인)	
본인은 위의 번호로 공고(지명통지)한 귀 대학의 일반(제한, 지명)경쟁 입찰에 참가하고자 귀 대학에서 정한 문서(물품구매, 용역)입찰공고사항 및 입찰유의서 등을 모두 숙독하고 불일서류를 첨부하여 입찰 참가신청을 합니다.				
불일서류 : 기타 공고로서 정한 서류				
2021 년 월 일				

5-1. HWP 파일 처리에 대한 분석 및 한계 고찰

>>> hwp: 폐쇄적인 바이너리 구조로 인한 낮은 호환성

진행 과정

1. hwp파일을 pdf로 변환하는 작업 자동화 시도

linux서버에서 libreOffice cli를 이용 hwp => pdf로 변환

→ 여러가지(load, perm, ...) 이유로 실패

```
shell> libreoffice --headless --convert-to pdf "example.hwp"  
Error: source file could not be loaded
```

```
pip install pyhwp
```

2. pyhwp이용 hwp => html 변환

pip로 설치하지만 shell에서 실행시키기에 python으로 제어 불가능.

→ html, 표, 이미지 추출 → 웹브라우저에서 원본과 비슷하게 확인.

웹에서 사용하지 않는 이미지(bmp, tif), hwp제작시의 이미지 포맷

계획표(진행상황, 주간/월간)를 배경색으로 처리한경우

html의 class, css파일만으로 추출 불가.

이미지 OCR 실행(gif, jpg, png) => 기대이하의 성능, 의미전달이 안됨

```
# 1. HWP -> HTML 변환 (표 구조 유지를 위해 hwp5html 사용)  
# 이미지 파일들도 temp_dir에 자동으로 추출 => 웹 운영시 미리보기 가능  
try:  
    subprocess.run(["hwp5html", "--output", temp_dir, hwp_path], check=True)  
except Exception as e:  
    return f"[에러] HTML 변환 실패: {e}"  
  
# 2. BeautifulSoup이용해서 HTML 파싱 (텍스트 및 표 추출)  
html_file = os.path.join(temp_dir, "index.xhtml") # 보통 index.xhtml로 생성됨  
with open(html_file, "r", encoding="utf-8") as f:  
    soup = BeautifulSoup(f, "lxml")
```

결론

- html로 추출은 정상 작동하기에 웹에서 파일 미리보기 용도로 사용 가능.
- OCR의 성능이 뛰어나지 않고 기존 RAG 파이프라인에 결합하기 어려운 구조 때문에 외부 프로그램을 활용한 hwp파일 처리는 포기
- hwp 확장자인 원본 데이터를 외부에서 pdf로 변환해서 사용하기로 결정

5-2. PDF 파일 처리시 멀티모달 로더 작동 과정

```
class MultiModalLoader(BaseRFPDocumentLoader):
    def __init__(self):
        app_cfg = get_app_config()
        self.cfg = app_cfg.loader_config

        # ✅ image_processing 설정 반영
        self.ip = app_cfg.loader_config.image_processing
        Path(self.ip.image_output_dir).mkdir(parents=True, exist_ok=True)

        # 이미지 -> 텍스트 요약 모듈 초기화
        self.image_to_docs = ImageToDocs()

        self.TABLE_BLOCK_RE = re.compile(
            r"""
            (
                # table block start
                ^\|.*\|\s*$\n
                # header row
                ^\|(?:\s*:?-+:?\s*\|)+\s*$\n
                # separator row: |---| or |:---:|
                (?:\^\\|.*\|\s*$\n?)+
                # body rows (1+)
            )
            # table block end
            """,
            re.MULTILINE | re.VERBOSE,
        )

    def load(self, pdf_path: str | Path) -> List[Document]:
        pdf_path = Path(pdf_path)
        docs: List[Document] = []

        # ✅ 텍스트 추출
        docs.extend(self._extract_text_docs(pdf_path))

        # ✅ 테이블 추출
        if self.cfg.extract_tables:
            docs.extend(self._extract_table_docs(pdf_path))

        # ✅ 이미지 추출
        if self.ip.extract_images:
            docs.extend(self._extract_image_docs(pdf_path))

        return docs
```

멀티모달 로더 클래스 설계

- 파일 내에 있는 텍스트, 테이블, 이미지 등 다양한 형태의 데이터를 단독으로 처리할 수 있는 로더 설계

텍스트, 테이블, 이미지 추출

- 하나의 파일에서 텍스트, 테이블, 이미지 요소를 각각 추출
- 추출한 데이터를 LangChain에서 제공하는 Document타입의 데이터로 변환
 - 실제 데이터 및 메타데이터 저장 용이

5-2. PDF 파일 처리시 멀티모달 로더 작동 과정

파일에서 텍스트, 테이블, 이미지 요소 추출

```
def _extract_text_docs(self, pdf_path: Path) -> List[Document]:
    """
    PDF 파일에서 fitz로 텍스트를 추출하여 Document로 변환합니다.
    Args:
        pdf_path: PDF 파일 경로
    Returns:
        out: Document의 목록
    """
    out: List[Document] = []
    with fitz.open(pdf_path) as doc:
        end = (
            min(doc.page_count, self.cfg.max_pages)
            if self.cfg.max_pages
            else doc.page_count
        )
        for i in range(end):
            page = doc.load_page(i)
            text = page.get_text("text").strip()
            if not text:
                continue
            out.append(
                Document(
                    page_content=text,
                    metadata={
                        "source": str(pdf_path),
                        "page": i + 1,
                        "type": "text",
                    },
                )
            )
    return out
```

텍스트 추출

- fitz 패키지로 페이지별 텍스트 추출
- 추출된 텍스트를 page_content로 사용

메타데이터 구성

- 파일 경로, 추출된 페이지, 데이터 타입(text) 저장

5-2. PDF 파일 처리시 멀티모달 로더 작동 과정

```
def _extract_table_docs(self, pdf_path: Path) -> List[Document]:
    """
    PDF 파일에서 PyMuPDFLoader로 테이블을 추출하여 Document로 변환합니다.
    Args:
        pdf_path: PDF 파일 경로
    Returns:
        out: Document의 목록
    """
    out: List[Document] = []
    loader = PyMuPDFLoader(pdf_path, extract_tables="markdown")
    documents = loader.load()

    for idx, doc in enumerate(documents):
        table = self._split_md_tables(doc)
        if len(table) != 0:
            out.append(
                Document(
                    page_content=table[0],
                    metadata={
                        "source": str(pdf_path),
                        "type": "table",
                        "page": idx + 1,
                    },
                )
            )
    return out

def _split_md_tables(self, doc: Document):
    """
    Document에서 마크다운 형식의 테이블을 추출합니다.
    Args:
        doc: Document 객체
    Returns:
        tables: 마크다운 형식으로 추출된 테이블
    """
    content = doc.page_content
    tables: list[str] = self.TABLE_BLOCK_RE.findall(content) # list[str]

    return [t.strip() for t in tables]
```

테이블 추출

- PyMuPDFLoader 사용
- 파일 로드시 텍스트와 마크다운 형태의 테이블이 같은 텍스트 스트링에 묶여서 로드됨
- 클래스에 정의해놓은 마크다운 표현식을 이용해 마크다운 형태의 테이블을 텍스트에서 분리해 page_content로 사용

메타데이터 구성

- 파일 경로, 추출된 페이지, 데이터 타입(table) 저장

5-2. PDF 파일 처리시 멀티모달 로더 작동 과정

```
def _extract_image_docs(self, pdf_path: Path) -> List[Document]:  
    out: List[Document] = []  
    with fitz.open(pdf_path) as doc:  
        end = (  
            min(doc.page_count, self.cfg.max_pages)  
            if self.cfg.max_pages  
            else doc.page_count  
        )  
        for i in range(end):  
            page = doc.load_page(i)  
            images = page.get_images(full=True)  
            for j, img in enumerate(images):  
                xref = img[0]  
                base = doc.extract_image(xref)  
                img_bytes = base["image"]  
                ext = base.get("ext", "png")  
                img_file = (  
                    Path(self.ip.image_output_dir)  
                    / f"{pdf_path.stem}"  
                    / f"{pdf_path.stem}_p{i+1}_img{j+1}.{ext}"  
                )  
                img_file.parent.mkdir(parents=True, exist_ok=True)  
                if not img_file.exists():  
                    img_file.write_bytes(img_bytes)  
                # ✅ 핵심: 이미지 파일 → 캡션(한국어) Document 생성  
                out.extend(  
                    self.image_to_docs.make_docs_from_image(  
                        image_path=img_file,  
                        source=str(pdf_path),  
                        page=i + 1,  
                        extra_meta={"image_index": j + 1},  
                    )  
                )  
    return out
```

파일 내 이미지 추출

- fitz 패키지를 활용해 파일 내 페이지마다 존재하는 이미지를 읽음
- 읽은 이미지를 별도의 경로에 저장
- 로더 클래스에 이미지 캡션(요약 텍스트) 생성을 위한 클래스 객체가 멤버 변수로 존재
- 이미지 처리를 위한 객체에서 저장된 이미지 경로와 메타데이터를 입력으로 받아 각 이미지에 대한 요약 텍스트를 생성하는 make_docs_from_image 메소드 호출

5-2. PDF 파일 처리시 멀티모달 로더 작동 과정

```
def make_docs_from_image(
    self,
    image_path: str | Path,
    source: str,
    page: Optional[int] = None,
    extra_meta: Optional[Dict[str, Any]] = None,
) -> List[Document]:
    """
    이미지에서 캡션을 추출하여 Document로 변환합니다.
    Args:
        image_path: 이미지 파일 경로
        source: 원본 문서의 소스 정보
        page: 원본 문서의 페이지 번호
        extra_meta: 추가 메타데이터
    Returns:
        List[Document]: Document 목록
    """
    image_path = Path(image_path)
    meta = {"source": source, "type": "image", "image_path": str(image_path)}
    if page is not None:
        meta["page"] = page
    if extra_meta:
        meta.update(extra_meta)

    caption_text = (
        self._run_openai_caption_ko(image_path) if self.caption_cfg.enabled else ""
    )

    parts = [f"[IMAGE] {image_path.name}"]
    if caption_text:
        parts.append("[CAPTION_KO]\n" + caption_text)
    return [Document(page_content="\n\n".join(parts).strip(), metadata=meta)]
```

이미지에 대한 요약 텍스트 생성

- 입력 받은 파일 경로를 LLM의 입력으로 넣을 수 있는 형식으로 변환
- 이미지를 입력받은 LLM이 해당 이미지에 대한 요약 텍스트를 생성하면 해당 요약 텍스트와 메타데이터를 하나의 텍스트로 합쳐서 요약 텍스트를 구성함
- 요약 텍스트로 page_content를 구성

메타데이터 구성

- 이미지가 저장된 파일 경로, 이미지가 추출된 페이지, 데이터 타입(image) 저장

5-2. PDF 파일 처리시 멀티모달 로더 작동 과정

```
from __future__ import annotations

from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_core.documents import Document
from typing import List
from ..config import get_app_config

def split_documents(docs: List[Document]) -> List[Document]:
    """
    - text: chunking 적용
    - image: (보통 짧으니) 기본적으로 chunking 하지 않음 (원하면 옵션으로 확장 가능)
    - table: Markdown 테이블 구조 깨지므로 chunking 제외
    Args:
        docs: Document의 list
    Returns:
        out: chunking이 적용된 Document의 list
    """
    cfg = get_app_config()
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=cfg.chunking.chunk_size,
        chunk_overlap=cfg.chunking.chunk_overlap,
    )

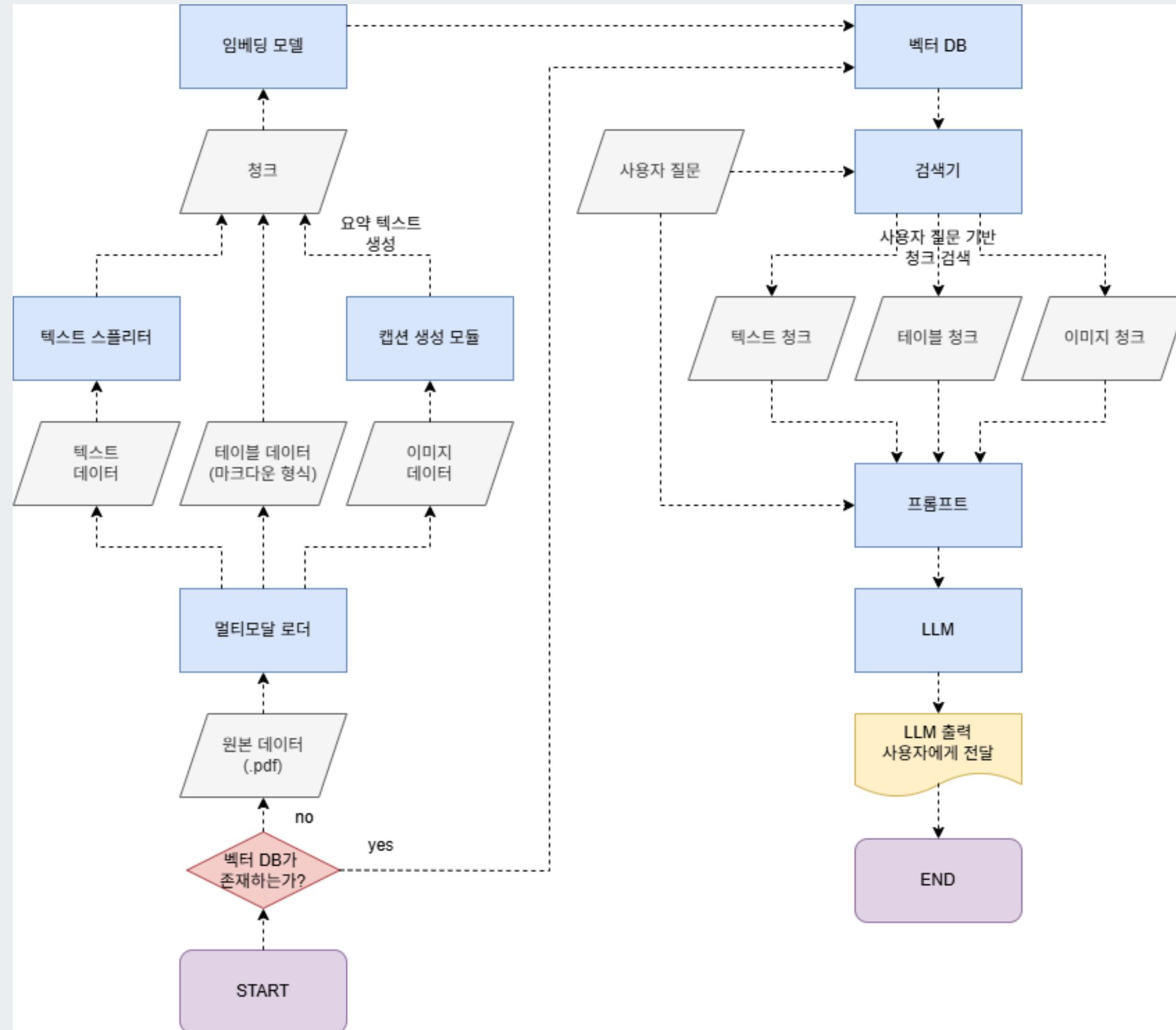
    out: List[Document] = []
    for d in docs:
        dtype = (d.metadata or {}).get("type", "text")
        if dtype in ["table", "table_error"]:
            out.append(d)
        elif dtype == "image":
            # 이미지 문서는 캡션이 이미 구조화되어 있으므로 그대로 유지
            out.append(d)
        else:
            out.extend(splitter.split_documents([d]))

    return out
```

데이터 형태에 따른 텍스트 스플리터 적용 방식

- 텍스트, 테이블, 이미지를 각각 추출했을 때 형태가 다름
- 텍스트의 경우 한 페이지에 존재하는 텍스트를 모두 읽어들이는 형태이므로, 텍스트 스플리터로 일정한 청크 사이즈로 나누어줌
- 테이블의 경우 페이지에 존재하는 테이블 단위로 구조화되어 있으므로 청킹하지 않고 유지
- 이미지의 경우 이미지 단위로 캡션이 생성되어 있으므로 청킹하지 않고 유지

5-3. 멀티모달 로더가 적용된 RAG 시스템 아키텍처 소개



6. RAG 시스템과 프론트엔드 연결

```
# [1] 앱 생성 및 수명주기 연결
# FastAPI 앱을 만드는데, "이 앱의 시작과 끝은 lifespan 함수가 관리한다"라고 지정해줍니다.
app = FastAPI(lifespan=lifespan)

# [2] 정적 파일 연결 (Mounting)
# "/static"이라는 주소로 들어오는 요청은 static_dir 폴더의 파일을 그대로 보여주라는 뜻입니다.
# 예: 브라우저가 http://.../static/style.css 를 요청하면 -> static 폴더의 style.css를 줌.
app.mount("/static", StaticFiles(directory=static_dir), name="static")

@app.get("/", response_class=HTMLResponse)
async def read_root():
    # [1] index.html 파일 찾기
    index_file = static_dir / "index.html"

    # [2] 파일 존재 여부 방어 코드
    # 만약 index.html이 없으면 404 에러를 띄웁니다.
    if not index_file.exists():
        return HTMLResponse(content="<h1>Error: index.html not found</h1>", status_code=404)

    # [3] 파일 읽어서 돌려주기
    # 파일을 열어서(open) 그 안의 HTML 텍스트를 읽은 뒤(read),
    # 브라우저에게 그대로 던져줍니다(return). 브라우저는 이 HTML을 해석해서 화면을 그립니다.
    with open(index_file, "r", encoding="utf-8") as f:
        return f.read()

# [1] 요청 모델 정의 (Pydantic)
# request: QuestionRequest -> "들어오는 데이터는 무조건 QuestionRequest(schemas.py) 모양이어야 해"
# 만약 사용자가 이상한 데이터를 보내면 FastAPI가 알아서 에러를 뱉습니다.
@app.post("/api/chat")
```

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>AI 지식상담 챗봇</title>

  <link href="https://fonts.googleapis.com/css2?family=Noto+Sans+KR:wght@300;400;500;700&display=swap" rel="stylesheet">
  <link rel="stylesheet" href="/static/css/style.css">
</head>
<body>
  <div class="app-container">
    <header class="chat-header">
      <div class="header-icon">
        <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-linecap="round" stroke-linejoin="round">
          <path d="M21 15a2 2 0 0 1-2 2H7l-4 4V5a2 2 0 0 1 2-2h14a2 2 0 0 1 2 2z"></path>
        </svg>
      </div>
      <div class="header-text">
        <h1>인공지능 챗봇 AI</h1>
        <p>인공지능 전문 컨설팅 AI</p>
      </div>
    </header>

    <main class="chat-container">
      <div id="chatBox" class="chat-box">
      </div>
    </main>

    <footer class="input-container">
      <div class="input-wrapper">
        <input type="text" id="chatInput" placeholder="질문을 입력해 주세요." autocomplete="off">
        <button id="sendBtn">전송</button>
      </div>
    </footer>
  </div>

  <script type="module" src="/static/js/main.js"></script>
</body>
</html>
```

프론트엔드 연동

- static 안의 index.html을 연동

CSS, JS 연동

- 디자인: static/ ccs/style.css
- 이벤트: static/js/main.js 연동

6. RAG 시스템과 프론트엔드 연결

```
import { chatAPI } from './api.js';
import { chatUI } from './ui.js';

async function handleSend() {
  const question = chatUI.getInput();
  if (!question) return;

  chatUI.appendMessage(question, "user");
  chatUI.toggleInput(true);
  chatUI.appendMessage("", "bot", true);

  try {
    const data = await chatAPI.sendMessage(question);
    chatUI.removeLoading();
    chatUI.appendMessage(data.answer, "bot");
  } catch (e) {
    chatUI.removeLoading();
    chatUI.appendMessage("오류가 발생했습니다.", "bot");
  } finally {
    chatUI.toggleInput(false);
  }
}
```

```
1 // 서버 통신 로직
2 export const chatAPI = {
3   async sendMessage(query) {
4     try {
5       1 const response = await fetch("/api/chat", {
6         method: "POST",
7         headers: { "Content-Type": "application/json" },
8         body: JSON.stringify({ query: query })
9       });
10
11       if (!response.ok) throw new Error("Server Error");
12       2 return await response.json();
13     } catch (error) {
14       console.error(error);
15       throw error;
16     }
17   }
18 };
19
```

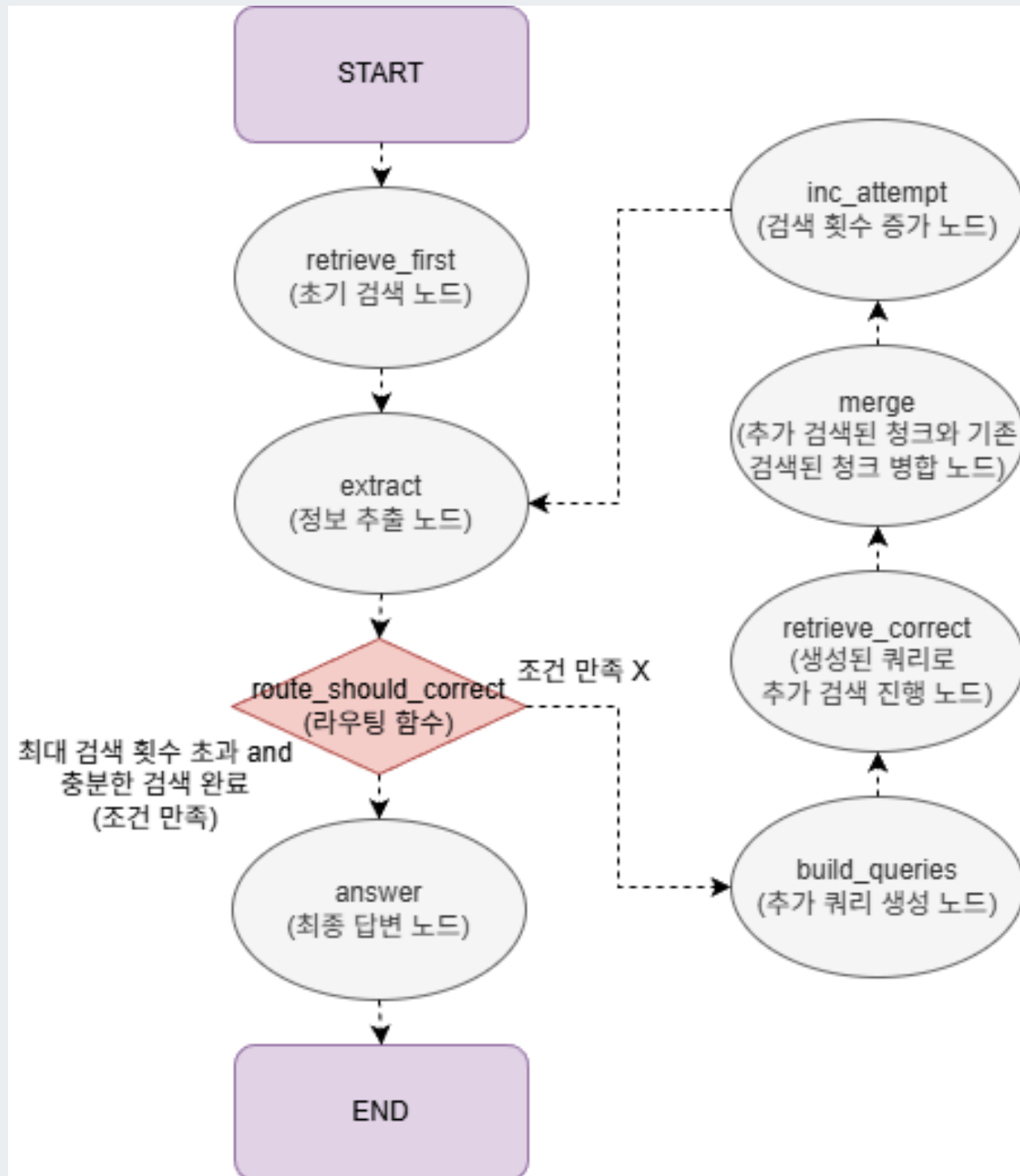
프론트엔드 연동

- static/js/main.js에서 ui.js, api.js 연동
- api.js에서 api/chat를 통해 서버와 통신

서버와 소통

1. api/chat으로 입력 받은 데이터를 json의 형태로 송신 및 수신
 - fetch: 서버로 송신
 - await: 답변 기다리기
 - cost response: 답변 수신
2. 수신된 문자열 데이터 실행 가능한 객체 데이터로 변환

7. CRAG를 적용한 아키텍처 소개



CRAG 아키텍처 흐름

- 그래프 상태로 **검색된 청크**와 LLM이 청크에서 추출한 **필수 정보 목록** (공고 번호, 공고 차수, 사업명, 사업 금액, 발주 기관, 공개 일자, 입찰 참여 시작일, 입찰 참여 마감일) 유지
- 검색된 청크에서 LLM이 필수 정보 목록을 모두 추출했거나 설정해둔 최대 검색 횟수를 넘기면 추출된 정보를 활용해 답변 생성
- 만약 필수 정보들이 모두 추출되지 않았거나 최대 검색 횟수를 채우지 못한 경우 CRAG 파이프라인 수행
- 사용자 질문 기반 추가 쿼리 생성, 생성된 쿼리 기반 청크 재검색 및 필수 정보 목록 추출, 기존 정보와 병합

8. 시스템 성능 평가

RAG 시스템의 정량적 평가 방법

- RAG 시스템의 정량적 평가는 검색 단계, 생성 단계에서 이루어질 수 있음
- 검색 단계의 평가는 사용자의 질문과 관련된 청크를 검색기가 잘 검색했는지를 측정하는 지표인 MRR(Mean Reciprocal Rank)를 측정하여 이루어짐
 - 이를 측정하기 위해 질문, 질문과 관련된 청크가 쌍을 이루는 평가용 데이터셋이 필요
- 생성 단계의 평가는 생성된 답변이 질문과 얼마나 관련 있고 제공된 컨텍스트에 기반했는지 등을 LLM이 평가하는 LLM-as-a-judge 방식으로 진행할 수 있고, 또 정답 텍스트와 LLM의 답변 사이 유사도를 BLEU, ROUGE 등의 지표로 측정해 평가할 수도 있음
 - 이를 위해서는 원본 데이터를 기반으로 한 질문/답변 쌍 평가용 데이터셋이 필요

8. 시스템 성능 평가

정량적 평가 방식의 한계

- 주어진 평가용 데이터셋이 없었기에 LLM을 활용해 데이터셋을 만드는 방법을 시도
- 만들어진 평가용 데이터셋의 품질이 매우 좋지 않았음
 - 질문이 명확하지 않음
 - 질문의 다양성이 현저히 떨어짐
 - 데이터셋 생성 또한 RAG를 기반으로 하기에 청크를 기반으로 데이터셋을 생성하므로 주요 내용이 누락되는 경우가 발생
- 평가용 데이터셋의 품질이 좋지 않아서 LLM-as-a-judge 평가를 해보아도 의미있는 결과가 나오지 않음
- 정량적 평가 방식 대신 다양한 질문을 통해 정성적 평가를 진행

```
{
  "question": "이 제안요청서의 사업명(프로젝트명), 주관 기관(대상 기관), 예산, 제출 방식은 무엇인가?",
  "reference_answer": "사업명: 종합정보시스템 및 홈페이지 등 구축 용역;
주관기관: 2025 구미아시아육상경기선수권대회 조직위원회;
예산: 문서에 정보가 없음;
제출 방식: 문서에 정보가 없음",
  "metadata": {
    "pdf_path": "eval/data/raw_pdf/2025 구미 아시아육상경기선수권대회 조직위원회_2025 구미아시아육상경.pdf",
    "page": 1,
    "type": "text",
    "chunk_index": 0,
    "target_field": "other",
    "evidence": "종합정보시스템 및 홈페이지 등 구축 용역 제안요청서 /
  ㅇ (주 관) 2025 구미아시아육상경기선수권대회 조직위원회",
    "difficulty": "medium"
  }
}

{
  "question": "이 용역의 주요 요구사항, 대상 기관(수요기관), 예산, 제출 방식은 무엇인가?",
  "reference_answer": "주요 요구사항: 아시아육상대회를 위한 종합정보시스템 구축 및 운영;
종합정보시스템 운영을 위한 대회 정보화 시설 구축; 경기 결과와 계측기, 전광판, 방송 장비, 인터넷 정보시스템의 연동;
종합정보시스템을 연계한 아시아육상대회 홈페이지 구축·관리.
대상 기관: 문서에 정보가 없음.
예산: 260,000,000원 ※ 부가가치세 포함.
제출 방식: 문서에 정보가 없음.",
  "metadata": {
    "pdf_path": "eval/data/raw_pdf/2025 구미 아시아육상경기선수권대회 조직위원회_2025 구미아시아육상경.pdf",
    "page": 2,
    "type": "text",
    "chunk_index": 0,
    "target_field": "other",
    "evidence": "ㅇ (사 업 내 용) 아시아육상대회를 위한 종합정보시스템 구축 및 운영;
종합정보시스템 운영을 위한 대회 정보화 시설 구축;
경기 결과와 계측기, 전광판, 방송 장비, 인터넷 정보시스템의 연동;
종합정보시스템을 연계한 아시아육상대회 홈페이지 구축·관리 /
  ㅇ (사 업 비) 260,000,000원 ※ 부가가치세 포함",
    "difficulty": "medium"
  }
}
```


8. 시스템 성능 평가 - 정성적 평가

고려대학교 차세대 포털 시스템 사업의 개요에 대해서 알려줘

개요는 다음과 같습니다.

- 사업명: 고려대학교 차세대 포털·학사 정보시스템 구축 사업
- 사업기간: 계약일로부터 24개월 이내(추가로 사업종료일로부터 무상유지보수 12개월)
- 예산: 11,270,000,000원(VAT 포함, 3년 분할지급 — 2024 약 30%, 2025 약 40%, 2026 약 30%)
- 입찰/계약방법: 제한경쟁 입찰(협상에 의한 계약)
- 추진 배경 및 필요성:
- 학령인구 감소, 코로나·AI 등 교육환경 급변에 대응 필요
- 분산·노후화된 시스템과 데이터의 통합 필요
- 데이터 기반 대학경영 지원 및 의사결정 개선 필요
- 포털의 정보 접근성·유용성 개선을 통한 대학 경쟁력 강화
- 사업 범위(주요 시스템 및 기능)

1. 포털시스템

- 통합로그인(SSO), 통합·지능형 검색, 마이페이지, 공지/알림, 일정관리, 커뮤니티·게시판, 위젯(개인화) 등

1. 포털시스템

- 통합로그인(SSO), 통합·지능형 검색, 마이페이지, 공지/알림, 일정관리, 커뮤니티·게시판, 위젯(개인화) 등
- 웹메일·챗봇·전자결재·학사/행정 서비스 등 연계
- 학생(졸업생 포함), 교직원, 연구원 대상, 신분·개인별 주요정보 제공
- 간편인증 및 2차인증 적용, 사용자별 검색 우선노출·검색 결과 추천

2. 모바일서비스

- 통합로그인, 통합/지능형 검색, 마이페이지, 공지/알림, 일정관리, 공간·기기 예약, 맞춤·추천정보, 학사·행정정보 등
- 기존 보유 앱(호잇)을 하이브리드 방식으로 재구축(일부 서비스 추가/삭제), 모바일 웹 포털 및 푸시 기능 포함

3. 학사행정시스템

- 기준정보·권한·사용자 관리 등 공통관리, 학적·교과·수업·성적·졸업·교직·등록·장학·병무·보건·공제·국제어학원 등 학사 전반 기능
- 학생 데이터 통합(학부·대학원·의학계열 포함), 수강신청 시스템(모바일 포함)
- KUchive(비교과/역량), LMS, 도서관, 기숙사, 현장실습, CossHUB 등 유관시스템 연계
- 아키텍처·연계
- 다계층 아키텍처(PC/태블릿/스마트폰 채널 → 통합로그인/2차인증 → 포털/모바일/홈페이지/UMS 등)

사·행정정보 등

- 기존 보유 앱(호잇)을 하이브리드 방식으로 재구축(일부 서비스 추가/삭제), 모바일 웹 포털 및 푸시 기능 포함

3. 학사행정시스템

- 기준정보·권한·사용자 관리 등 공통관리, 학적·교과·수업·성적·졸업·교직·등록·장학·병무·보건·공제·국제어학원 등 학사 전반 기능
- 학생 데이터 통합(학부·대학원·의학계열 포함), 수강신청 시스템(모바일 포함)
- KUchive(비교과/역량), LMS, 도서관, 기숙사, 현장실습, CossHUB 등 유관시스템 연계
- 아키텍처·연계
- 다계층 아키텍처(PC/태블릿/스마트폰 채널 → 통합로그인/2차인증 → 포털/모바일/홈페이지/UMS 등)
- 중앙에 경영통계(BI), 데이터포털(Search·정보복득·분석), AI 기반 추천/분석 기능
- 내부 시스템(K-Class, LMS 등) 및 외부 기관(교육부·연구재단·국세청·은행 등) 연계 포함
- 기타
- 본 사업은 3개년에 걸친 다년사업으로 대금도 3년에 걸쳐 분할 지급됨

문서에 근거한 주요 내용만 요약했습니다.

질문과 관련된 청크를 잘 검색한 경우

- 검색된 청크를 기반으로 사용자의 질문에 대한 주요 사항들을 정리해 잘 알려줌
- 검색된 청크를 기반으로 답변을 생성하기 때문에 청크에 사용자가 원하는 정보가 없을 경우 해당 부분이 누락된 상태로 답변이 생성될 수 있음

8. 시스템 성능 평가 - 정성적 평가

입찰메이트 AI
입찰지원 전문 컨설팅 AI

필요하시면
- 전체 RFP 원문 또는 세부 요구사항(요구기능 목록, 일정표, 예산표 등)이 포함된 페이지를 추가로 올려주시면 누락된 항목(예산·일정·세부기능·평가기준 등)을 더 정확히 요약해 드리겠습니다.

그럼 한영 대학에서 발주한 사업의 사업 금액은 얼마야?

문서에 사업 금액 정보가 없습니다.

제공된 발제문에는 입찰가격제안서 양식(«총괄 : 금 ____ 원정(W ____)» 칸)이 포함되어 있으나 실제 금액은 기재되어 있지 않습니다. 즉, 이 RFP 발제본에서는 사업 총예산(금액)이 명시되어 있지 않습니다.

원하시면 다음을 도와드리겠습니다.
- RFP의 다른 페이지(예산표·공고문 등)를 올려주시면 금액을 찾아 드리겠습니다.
- 입찰공고(기관 공고 게시판) 확인 방법을 안내해 드리겠습니다.

질문을 입력해 주세요.

전송

검색 청크에 질문과 관련된 정보가 없는 경우

- 질문을 입력했을 때 검색기가 질문과 관련된 청크를 찾지 못하는 경우가 발생
- page1에 사업예산이 130,000,000원으로 되어 있지만, page37의 입찰가격 제안서에서 금액(부가세포함)에 해당하는 빈 값을 찾음
- “트랙제 교육과정 운영프로그램 개발”과 같은 특정 단어를 포함해 질문을 했을 때 질문 쿼리와 청크간 벡터 유사도가 높아져 사업 금액과 관련된 청크를 찾기도 함

8. 시스템 성능 평가 - 정성적 평가



검색 청크에 민감한 정보가 포함된 경우

전화번호, 생년월일 등 개인정보가 포함되거나 법률상 민감한 정보가 청크에 포함된 경우 LLM이 응답을 거절해 응답이 출력되지 않음

입
찰
자|상호 또는 법인명칭||법인등록번호|법인등록번호||
입
찰
자|주 소||전 화 번 호|전 화 번 호||
입
찰
자|대 표 자||생 년 월 일|생 년 월 일||
본인은 국가를 당사자로 하는 계약에 관한 법률시행령 제33조에 의거 발주된 본 사업의
며, 귀 기관에서 정한 사업 기간 내에 사업을 성실하
게 수행할 것을 약속하며 본 가
1부
2024년 월 일
입찰자 :
를 당사자로 하는 계약에 관한 법률시행령 제33조에 의거 발주된 본 사업의 제안
서 정한 사업 기간 내에 사업을 성실하
게 수행할 것을 약속하며 본 가격제안입찰서

프로젝트 개선 필요 사항

CRAG 아키텍처 적용

- CRAG 아키텍처를 적용해 사용자의 질문과 관련된 정보를 모두 찾을 때까지 검색기가 반복 검색을 진행하고 정보를 누적시켜 활용하는 구조를 구현해 LLM이 생성하는 답변의 품질 상승을 기대할 수 있음

정량적 평가용 데이터셋 개선

- 좋은 품질의 검색 단계 평가용 데이터셋과 생성 단계 평가용 데이터셋을 만들면 신뢰도 높은 정량적 RAG 시스템 평가가 가능해짐

CSV 파일 정보 활용

- LangGraph에서 실행 흐름 분기를 통해 사용자 질문에 대해 RAG 시스템을 수행했을 때 유의미한 답변을 생성하지 못했을 경우, CSV 파일에 저장되어 있는 파일 별 주요 요약 정보를 활용해 LLM이 답변을 생성하도록 해 RAG 시스템의 검색 성능을 보완해줄 수 있음

파일 로드 단계에서 중복 청크 제거

- 현재 멀티모달 로더에서 읽어들이는 텍스트에는 테이블 내 텍스트도 포함됨
- 테이블과 중복되는 부분을 제거해 중복 청크를 줄여 검색기의 성능 향상 가능

민감한 정보 보호를 위한 데이터 전처리

- 프롬프트에서 청크에 민감한 정보 포함 시 민감한 정보를 제외하고 답변을 생성하도록 프롬프트를 작성해도 개인정보가 포함된 청크 자체가 LLM의 입력으로 들어가면 LLM이 입력 생성을 안하는 경우가 생김
- 로더 단계에서 생성된 청크 내에 민감한 정보가 포함되어 있을 경우, 마스킹하거나 제거하는 로직을 구현할 필요 있음

검색 단계 고도화 기술 적용

- 검색 단계에서 Reranking, Hybrid Search 기법 적용 시 검색 품질 향상 가능

THANK YOU